

Started on	Saturday, 21 June 2025, 10:35 AM
State	Finished
Completed on	Saturday, 21 June 2025, 12:07 PM
Time taken	1 hour 31 mins
Marks	2.00/2.00
Grade	10.00 out of 10.00 (100%)

Viết chương trình đọc một **đơn đồ thị có hướng, có trọng số không âm** từ bàn phím và in ra chiều dài đường đi ngắn nhất từ đỉnh 1 đến đỉnh n.

Đầu vào (Input)

Dữ liệu đầu vào được nhập từ bàn phím với định dạng:

- Dòng đầu tiên chứa 2 số nguyên n và m ($1 \leq n < 100; 0 \leq m < 500$)
- m dòng tiếp theo mỗi dòng chứa 3 số nguyên u, v, w mô tả cung (u, v) có trọng số w ($0 \leq w \leq 100$).

Đầu ra (Output)

- In ra màn hình chiều dài của đường đi ngắn nhất từ 1 đến n. Nếu không có đường đi từ 1 đến n, in ra -1.
- Xem thêm ví dụ bên dưới.

Gợi ý

- Sau khi kết thúc thuật toán nếu $pi[u] = \infty$ thì có nghĩa là không có đường đi từ s đến n.

For example:

Input	Result
3 3 1 2 3 2 3 5 1 3 10	8
3 1 1 2 5	-1
6 9 1 2 7 1 3 9 1 5 14 2 3 10 2 4 15 3 4 11 3 5 2 4 6 6 5 6 9	20

Answer: (penalty regime: 10, 20, ... %)

```

1  #include <stdio.h>
2
3  #define MAX_SIZE 100
4
5  //-----Graph-----
6  #define NO_EDGE -1 // Vì là ma trận trọng số
7  typedef struct {
8      int n, m;
9      int A[MAX_SIZE][MAX_SIZE];
10 } Graph;
11
12 void init_graph(Graph* pG, int n) {
13     pG->n = n;
14     pG->m = 0;
15
16     for (int i = 0; i <= n; i++) {
17         for (int j = 0; j <= n; j++) {
18             pG->A[i][j] = NO_EDGE;
19         }
20     }
21 }
22
23 void add_edge(Graph* pG, int u, int v, int w) {
24     pG->A[u][v] = w;
25     pG->A[v][u] = w;
26     pG->m++;
27 }
28
29 int adj(Graph* pG, int u, int v) {
30     return pG->A[u][v] > NO_EDGE;

```

```

31 }
32
33 //-----Moore Dijkstra-----
34 #define oo 999999
35 #define NOT_SURE 0
36 #define SURE 1
37 int parent[MAX_SIZE];
38 int visited[MAX_SIZE];
39 int d[MAX_SIZE]; // Store length of path
40
41 void Moore_Dijkstra(Graph* pG, int s) {
42     for (int i = 0; i <= pG->n; i++) {
43         d[i] = oo;
44         visited[i] = NOT_SURE;
45     }
46
47     parent[s] = -1;
48     d[s] = 0;
49
50     // L?p n l?n (m?i l?n ch?n m?t đ?nh và đánh d?u nó là SURE)
51     for (int i = 1; i <= pG->n; i++) {
52         int u = -1;
53         int min_u = oo;
54         for (int v = 1; v <= pG->n; v++) {
55             if (visited[v] == NOT_SURE && d[v] < min_u) {
56                 min_u = d[v];
57                 u = v;
58             }
59         }
60
61         if (u == -1) // KhÃ'ng tÃ-m Ä'Æ°á»£c u
62             break;
63
64         visited[u] = SURE;
65
66         for (int v = 1; v <= pG->n; v++) {
67             int w = pG->A[u][v];
68             if (adj(pG, u, v) && d[u] + w < d[v]) {
69                 d[v] = d[u] + w;
70                 parent[v] = u;
71             }
72         }
73     }
74
75 }
76
77
78 int main() {
79     Graph G;
80     int n, m;
81     scanf("%d%d", &n, &m);
82     init_graph(&G, n);
83
84     for (int e = 0; e < m; e++) {
85         int u, v, w;
86         scanf("%d%d%d", &u, &v, &w);
87         add_edge(&G, u, v, w);
88     }
89
90     Moore_Dijkstra(&G, 1);
91     if (d[n] < oo)
92         printf("%d\n", d[n]);
93     else
94         printf("-1\n");
95 }

```

	Input	Expected	Got	
✓	3 3 1 2 3 2 3 5 1 3 10	8	8	✓

	Input	Expected	Got	
✓	3 1 1 2 5	-1	-1	✓
✓	6 9 1 2 7 1 3 9 1 5 14 2 3 10 2 4 15 3 4 11 3 5 2 4 6 6 5 6 9	20	20	✓
✓	6 7 1 2 7 1 3 9 1 5 14 2 3 10 2 4 15 3 4 11 3 5 2	-1	-1	✓
✓	6 9 1 2 7 1 3 9 1 5 14 2 3 10 2 4 15 3 4 11 3 5 3 4 6 6 5 6 9	21	21	✓

Passed all tests! ✓

Question author's solution (C):

```

1  #include <stdio.h>
2
3  #define MAXN 100
4  #define oo 999999
5  #define NO_EDGE -1
6
7
8  typedef struct {
9      int n, m;
10     int w[MAXN][MAXN];
11 } Graph;
12
13 void init_graph(Graph *pG, int n) {
14     pG->n = n;
15     pG->m = 0;
16     for (int u = 1; u <= n; u++)
17         for (int v = 1; v <= n; v++)
18             pG->w[u][v] = NO_EDGE;
19 }
20
21 void add_edge(Graph *pG, int u, int v, int w) {
22     pG->w[u][v] = w;
23 }
24
25 int mark[MAXN];
26 int pi[MAXN];
27 int p[MAXN];
28
29 void MooreDijkstra(Graph *pG, int s) {
30     int u, v, it;
31     for (u = 1; u <= pG->n; u++) {
32         pi[u] = oo;
33         mark[u] = 0;
34     }
35 }

```

```

36 pi[s] = 0; //chiều dài đường đi ngắn nhất từ s đến chính nó bằng 0
37 p[s] = -1; //trước đỉnh s không có đỉnh nào cả
38
39 //Lặp n-1 lần
40 for (it = 1; it < pG->n; it++) {
41     //1. Tìm u có mark[u] == 0 và có pi[u] nhỏ nhất
42     int j, min_pi = oo;
43     for (j = 1; j <= pG->n; j++)
44         if (mark[j] == 0 && pi[j] < min_pi) {
45             min_pi = pi[j];
46             u = j;
47         }
48     //2. Đánh dấu u đã xét
49     mark[u] = 1;
50
51     //3. Cập nhật pi và p của các đỉnh kề của v (nếu thoà)
52     for (v = 1; v <= pG->n; v++)
53         if (pG->W[u][v] != NO_EDGE && mark[v] == 0)
54             if (pi[u] + pG->W[u][v] < pi[v]) {
55                 pi[v] = pi[u] + pG->W[u][v];
56                 p[v] = u;
57             }
58     }
59 }
60
61
62 int main() {
63     Graph G;
64     int n, m;
65     scanf("%d%d", &n, &m);
66     init_graph(&G, n);
67
68     for (int e = 0; e < m; e++) {
69         int u, v, w;
70         scanf("%d%d%d", &u, &v, &w);
71         add_edge(&G, u, v, w);
72     }
73
74     MooreDijkstra(&G, 1);
75     if (pi[n] < oo)
76         printf("%d\n", pi[n]);
77     else
78         printf("-1\n");
79
80     return 0;
81 }

```

Correct

Marks for this submission: 1.00/1.00.

Viết chương trình đọc một **đơn đồ thị vô hướng, có trọng số không âm** từ bàn phím và in ra chiều dài đường đi ngắn nhất từ đỉnh 1 đến đỉnh n.

Đầu vào (Input)

Dữ liệu đầu vào được nhập từ bàn phím với định dạng:

- Dòng đầu tiên chứa 2 số nguyên n và m ($1 \leq n < 100; 0 \leq m < 500$)
- m dòng tiếp theo mỗi dòng chứa 3 số nguyên u, v, w mô tả cung (u, v) có trọng số w ($0 \leq w \leq 100$).

Đầu ra (Output)

- In ra màn hình chiều dài của đường đi ngắn nhất từ 1 đến n. Nếu không có đường đi từ 1 đến n, in ra -1.
- Xem thêm ví dụ bên dưới.

Gợi ý

- Sau khi kết thúc thuật toán nếu $pi[u] = \infty$ thì có nghĩa là không có đường đi từ s đến n.

For example:

Input	Result
3 3 2 1 3 3 2 4 1 3 10	7
3 1 3 2 5	-1
6 9 1 2 7 3 1 9 1 5 14 2 3 10 4 2 15 3 4 11 3 5 2 6 4 6 5 6 10	21

Answer: (penalty regime: 10, 20, ... %)

```

1  #include <stdio.h>
2
3  #define MAX_SIZE 100
4
5  //-----Graph-----
6  #define NO_EDGE -1 // Vì là ma trận trọng số
7  typedef struct {
8      int n, m;
9      int A[MAX_SIZE][MAX_SIZE];
10 } Graph;
11
12 void init_graph(Graph* pG, int n) {
13     pG->n = n;
14     pG->m = 0;
15
16     for (int i = 0; i <= n; i++) {
17         for (int j = 0; j <= n; j++) {
18             pG->A[i][j] = NO_EDGE;
19         }
20     }
21 }
22
23 void add_edge(Graph* pG, int u, int v, int w) {
24     pG->A[u][v] = w;
25     pG->A[v][u] = w;
26     pG->m++;
27 }
28
29 int adj(Graph* pG, int u, int v) {
30     return pG->A[u][v] > NO_EDGE;

```

```

31 }
32
33 //-----Moore Dijkstra-----
34 #define oo 999999
35 #define NOT_SURE 0
36 #define SURE 1
37 int parent[MAX_SIZE];
38 int visited[MAX_SIZE];
39 int d[MAX_SIZE]; // Store length of path
40
41 void Moore_Dijkstra(Graph* pG, int s) {
42     for (int i = 0; i <= pG->n; i++) {
43         d[i] = oo;
44         visited[i] = NOT_SURE;
45     }
46
47     parent[s] = -1;
48     d[s] = 0;
49
50     // L?p n l?n (m?i l?n ch?n m?t đ?nh và đánh d?u nó là SURE)
51     for (int i = 1; i <= pG->n; i++) {
52         int u = -1;
53         int min_u = oo;
54         for (int v = 1; v <= pG->n; v++) {
55             if (visited[v] == NOT_SURE && d[v] < min_u) {
56                 min_u = d[v];
57                 u = v;
58             }
59         }
60
61         if (u == -1) // KhÃ'ng tÃ-m Ä'Æ°á»£c u
62             break;
63
64         visited[u] = SURE;
65
66         for (int v = 1; v <= pG->n; v++) {
67             int w = pG->A[u][v];
68             if (adj(pG, u, v) && d[u] + w < d[v]) {
69                 d[v] = d[u] + w;
70                 parent[v] = u;
71             }
72         }
73     }
74
75 }
76
77
78 int main() {
79     Graph G;
80     int n, m;
81     scanf("%d%d", &n, &m);
82     init_graph(&G, n);
83
84     for (int e = 0; e < m; e++) {
85         int u, v, w;
86         scanf("%d%d%d", &u, &v, &w);
87         add_edge(&G, u, v, w);
88     }
89
90     Moore_Dijkstra(&G, 1);
91     if (d[n] < oo)
92         printf("%d\n", d[n]);
93     else
94         printf("-1\n");
95 }

```

	Input	Expected	Got	
✓	3 3 2 1 3 3 2 4 1 3 10	7	7	✓

	Input	Expected	Got	
✓	3 1 3 2 5	-1	-1	✓
✓	6 9 1 2 7 3 1 9 1 5 14 2 3 10 4 2 15 3 4 11 3 5 2 6 4 6 5 6 10	21	21	✓
✓	6 7 1 2 7 1 3 9 1 5 14 2 3 10 2 4 15 3 4 11 5 3 2	-1	-1	✓
✓	6 9 2 1 5 1 3 9 5 1 14 2 3 10 4 2 16 3 4 11 3 5 3 4 6 6 6 5 8	20	20	✓

Passed all tests! ✓

Question author's solution (C):

```

1  #include <stdio.h>
2
3  #define MAXN 100
4  #define oo 999999
5  #define NO_EDGE -1
6
7
8  typedef struct {
9      int n, m;
10     int w[MAXN][MAXN];
11 } Graph;
12
13 void init_graph(Graph *pG, int n) {
14     pG->n = n;
15     pG->m = 0;
16     for (int u = 1; u <= n; u++)
17         for (int v = 1; v <= n; v++)
18             pG->w[u][v] = NO_EDGE;
19 }
20
21 void add_edge(Graph *pG, int u, int v, int w) {
22     pG->w[u][v] = w;
23     pG->w[v][u] = w;
24     pG->m++;
25 }
26
27 int mark[MAXN];
28 int pi[MAXN];
29 int p[MAXN];
30
31 void MooreDijkstra(Graph *pG, int s) {
32     int u, v, it;
33     for (u = 1; u <= pG->n; u++) {
34         pi[u] = oo;
35         mark[u] = 0;

```

```

36     }
37
38     pi[s] = 0; //chiều dài đường đi ngắn nhất từ s đến chính nó bằng 0
39     p[s] = -1; //trước đỉnh s không có đỉnh nào cả
40
41     //Lặp n-1 lần
42     for (it = 1; it < pG->n; it++) {
43         //1. Tìm u có mark[u] == 0 và có pi[u] nhỏ nhất
44         int j, min_pi = oo;
45         for (j = 1; j <= pG->n; j++)
46             if (mark[j] == 0 && pi[j] < min_pi) {
47                 min_pi = pi[j];
48                 u = j;
49             }
50         //2. Đánh dấu u đã xét
51         mark[u] = 1;
52
53         //3. Cập nhật pi và p của các đỉnh kề của v (nếu thoả)
54         for (v = 1; v <= pG->n; v++)
55             if (pG->W[u][v] != NO_EDGE && mark[v] == 0)
56                 if (pi[u] + pG->W[u][v] < pi[v]) {
57                     pi[v] = pi[u] + pG->W[u][v];
58                     p[v] = u;
59                 }
60     }
61 }
62
63
64 int main() {
65     Graph G;
66     int n, m;
67     scanf("%d%d", &n, &m);
68     init_graph(&G, n);
69
70     for (int e = 0; e < m; e++) {
71         int u, v, w;
72         scanf("%d%d%d", &u, &v, &w);
73         add_edge(&G, u, v, w);
74     }
75
76     MooreDijkstra(&G, 1);
77     if (pi[n] < oo)
78         printf("%d\n", pi[n]);
79     else
80         printf("-1\n");
81
82     return 0;
83 }

```

Correct

Marks for this submission: 1.00/1.00.

◀ Bài tập 1 - Thuật toán Moore -
Dijkstra (pi và p)

Jump to...

Bài tập 3 - Thuật toán Moore -
Dijkstra (đường đi) ▶